Evanesca: Measuring DeFi Value Extraction at Scale

ANONYMOUS AUTHOR(S)

DeFi protocols have suffered over $10 billion in losses between 2020 and 2025, yet most incidents are studied as isolated exploits. We observe that many attacks share recurring structure at the level of financial actions such as swap, borrow, and repay, suggesting that value extraction can be measured systematically. We present Evanesca, a measurement pipeline that reconstructs Semantic Financial Graphs (SFGs) from on-chain activity and evaluates reusable constraints expressed in a domain-specific language. Applying nine constraints in 107 lines to 14,187,803 Ethereum transactions (2020–2024) and 40 confirmed incidents spanning five chains, Evanesca reconstructs every incident without incident-specific signatures. The same rule set flags 1,418 operational instances: 956 baseline arbitrage and 462 non-baseline instances spanning reward-distribution manipulation, price manipulation, yield-bearing token deviations, and a long tail of mixed signals. Source-code investigation of flagged instances uncovered 219 previously unreported implementation defects across two categories: derivative pricing errors and reward-distribution bias. Replayable evidence traces let Evanesca serve as a reusable starting point for longitudinal studies of DeFi risk.

Technical report note. This document repackages the IMC-version Evanesca manuscript as a public technical report for the open-source release. The body text, structure, and measurement results follow the IMC version. The release package publishes the compiled PDF in the public repository; LaTeX sources are retained in the development repository.

Additional Key Words and Phrases: Blockchain, Ethereum, Smart contract analysis, MEV, DeFi security

1 Introduction

Decentralized Finance (DeFi) suffered over $1.6 billion in losses in Q1 2025 alone [10], part of a multi-year cumulative loss trend across the ecosystem, yet incident response and academic studies still treat most attacks as isolated exploits. We observe that many incidents share a common structure at the level of *financial actions* such as swap, borrow, and repay: the "borrow–manipulate–profit" sequence behind the bZx attack [14, 15] recurs across different protocols, chains, and years [24]. This repetitiveness suggests that value extraction can be measured systematically rather than investigated case by case.

However, existing approaches do not exploit this recurring structure. First, current DeFi attack detection systems do not generalize to unseen patterns: runtime monitors [8, 13] are configured per deployment, while attack-hunting frameworks such as DeFiRanger, BlockEye, DeFort, and DeFiScope [18, 20–22] bind detection to predefined attack templates—resulting in low coverage of incidents at the protocol layer, such as oracle manipulation and permissionless cross-contract interactions [24]. Second, none of these systems produce replayable evidence traces, and many report only aggregate statistics that cannot be independently verified. They emit alerts but do not compute financial values at each step, so analysts cannot triage variants or reproduce findings across studies [8, 13, 21].

Beyond these technical limitations lies a more fundamental challenge. Value extraction in DeFi does not divide neatly into attacks and normal behavior. Maximal Extractable Value (MEV) studies [1, 7, 19] show that significant value can be extracted by reordering transactions within a block without a discrete software vulnerability [24], blurring the boundary between "attack" and "strategy." We therefore frame this as a measurement problem rather than intent classification.

Our measurement framing requires protocol-independent invariants over typed financial actions, unlike prior frameworks that bind detection to fixed attack templates. The underlying insight is that the same service-level invariants recur across DeFi protocols, allowing a small rule set over typed actions such as swap, borrow, and repay to capture both benign baseline arbitrage and more suspicious non-baseline patterns. Accordingly, Evanesca is a measurement pipeline of four composable stages, each defined by its input and output. *Semantic recovery* takes protocol-specific smart-contract events as input and emits uniform financial actions (swap, borrow, deposit, etc.). *SFG construction* assembles these actions into a *Semantic Financial Graph* whose edges carry typed actions

annotated with USD-normalized values. *Constraint evaluation* then applies nine protocol-independent DSL rules[1] that encode invariants common to entire DeFi service classes (e.g., value preservation across any swap, collateral requirements for any borrow). *Evidence extraction* materializes each constraint match as a replayable trace. Each match is a *flagged instance*, a measurement signal indicating a pattern worth investigating, not a verdict that the transaction was harmful.

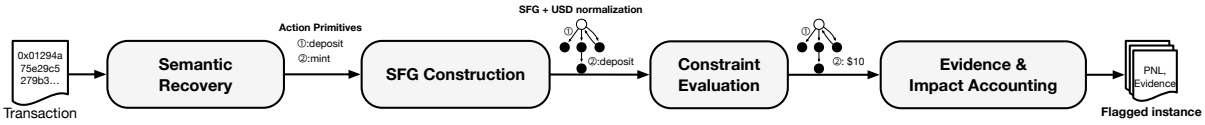

Fig. 1. Measurement pipeline. Evanesca performs semantic recovery, constructs USD-normalized SFGs, evaluates DSL constraints, and outputs replayable evidence traces.

In summary, we make the following contributions:

- **Multi-chain measurement at scale.** Using only general-purpose constraints (nine DSL rules in 107 lines, without incident-specific logic), we reconstruct all 40 confirmed incidents across five chains and flag 1,418 operational instances across five pattern categories over 14.2 M Ethereum transactions (2020–2024).
- **Reproducible methodology.** Every flagged instance is backed by a replayable evidence trace that records the full action sequence and profit path, enabling independent verification.
- **Previously unreported defects.** Source-code investigation of flagged instances uncovers 219 implementation defects across two categories: derivative pricing errors and reward-distribution bias.

2 Threat Model and Background

Threat model. Public blockchains maintain an append-only ledger of transactions readable by anyone. Smart contracts execute deterministically, making all financial activity publicly observable and reproducible. We study value-extraction patterns observable on this ledger: sequences of financial actions that violate invariants common to a DeFi service class, such as pool invariants for swaps, collateral requirements for borrows, or exchange-rate consistency for deposit/withdraw cycles, or that extract rewards and fees beyond intended use. A pattern is flagged when a constraint fires on the evidence trace. The flag is a measurement signal, not an intent label. We do *not* attempt real-world attribution. We measure patterns at the level of addresses and transaction sequences.

DeFi protocol primitives. Decentralized Finance (DeFi) protocols, autonomous financial services implemented as smart contracts, compose a small set of financial actions: Automated Market Makers (AMMs) price token *swaps* via pool invariants (e.g., $x \cdot y = k$ where x, y are token reserves); lending protocols let users *deposit* collateral, *borrow*, *repay*, and *withdraw*; *flash loans* provide uncollateralized capital that must be repaid within the same transaction; and *yield-bearing tokens* (e.g., Compound's cTokens, Aave's aTokens) are receipt tokens representing deposit positions whose exchange rate accrues over time. Price *oracles* supply external market data to contracts. Manipulating an oracle corrupts downstream pricing decisions. Every computation costs *gas*, priced in the native currency, and users pay gas fees proportional to transaction complexity. These primitives are the building blocks of both legitimate DeFi activity and the patterns we measure.

Value-Extraction Patterns. We define a *value-extraction pattern* as a repeatable sequence of *financial actions* executed to realize profit with minimal exposure. This definition is intentionally broad: routine arbitrage also fits and is retained as an explicit baseline in our evaluation. The distinction between baseline and non-baseline patterns is made during analysis: baseline patterns profit from existing price discrepancies across markets, whereas

[1]We use *rule* and *constraint* interchangeably throughout the paper. The collection of all nine forms our *rule set*.

non-baseline patterns require the actor to alter protocol state (e.g., inflating a token price in a low-liquidity pool) to create the profit opportunity.

Measurement objective. Given raw on-chain activity, we reconstruct the sequence of actions underlying each pattern, quantify counts and conservative impact, and output evidence artifacts that an analyst can inspect and reproduce. We do not classify adversarial intent, estimate total DeFi losses, or build a real-time monitoring system. Our focus is retrospective, evidence-based measurement.

3 Measurement Methodology

We operationalize the measurement objective as three research questions, then build a pipeline that answers them.

- **(RQ1)** Can general-purpose constraints reconstruct known DeFi incidents?
- **(RQ2)** What patterns emerge at scale?
- **(RQ3)** Do flagged patterns correspond to real implementation defects?

This section answers RQ1 and develops the pipeline shared by all three. Section 4 applies the same pipeline at scale for RQ2 and RQ3.

To answer these questions, we translate raw on-chain events into structured sequences of financial actions such as swaps, deposits, and borrows, with their associated values. Unlike contract-level analysis [6, 12, 17] or runtime monitoring [8, 13, 16], our approach operates on semantic financial actions that generalize across protocols and chains. Three challenges motivate our pipeline design.

(C1) Semantic gap: Protocols implement the same operation using different event schemas (e.g., `Mint` on Compound vs. `Deposit` on Aave); contract-specific heuristics therefore break across forks and upgrades. **(C2) Temporal composition:** Individual actions such as a swap or a borrow may appear normal in isolation. The extraction becomes visible only when the full ordered sequence within a transaction is analyzed together, often spanning multiple protocols. **(C3) Reproducibility:** Existing pipelines, whether rule-based monitors [8, 13] or template-based frameworks [21], typically emit alerts without the per-step computation needed for independent replay and triage.

Pipeline overview. Our pipeline processes on-chain data through four stages (Figure 1): semantic recovery, SFG construction, constraint evaluation, and evidence extraction. Each constraint match produces a *flagged instance*, which bundles the evidence trace from the SFG, the violated constraint, and per-entity PnL.

Semantic recovery. Protocol-specific adapters decode each transaction into five uniform action primitives: `swap`, `deposit`, `withdraw`, `borrow`, and `repay`. We cover 40+ protocol families spanning DEX, lending, flash-loan, and bridge services (Appendix B). Flash loans are emitted as a within-transaction `borrow+repay` pair. Because flash loans require no upfront capital, connecting borrowed funds to downstream profit is essential for identifying patterns that extract value with no upfront capital. Unlike contract-specific signatures, our adapters produce uniform output across protocol families, addressing C1.

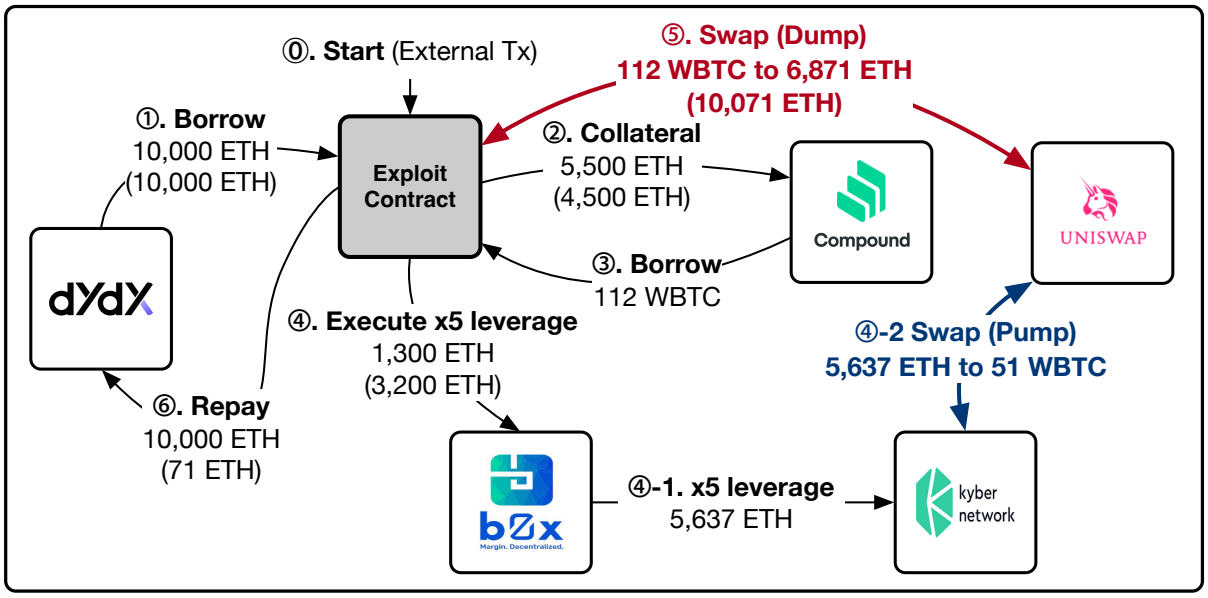

Fig. 2. Representative price manipulation pattern (bZx), the RQ1 walkthrough. Semantic recovery connects the manipulation step to profit across protocols.

SFG construction. We model each transaction as a directed graph $G = (V, E)$. Vertices represent entities such as accounts, liquidity pools, and lending markets, annotated with their protocol role. Edges represent typed financial actions, recording input/output assets, USD-normalized values, and the originating protocol. We convert token amounts to USD using the CoinGecko price API [4] at each transaction's block timestamp. Edges are ordered by transaction index, ensuring deterministic evaluation. This representation abstracts away protocol-specific event schemas while preserving the ordering and cross-protocol connectivity of financial actions. As a result, the same multi-step extraction pattern remains visible even when implemented by different contracts, addressing C1 and enabling C2.

Constraint evaluation. We express value-extraction patterns as reusable semantic rules over SFG edges. For each edge e whose input has nonzero USD value, we compute:

$$\rho(e) = \frac{\text{valueUSD}(\text{out}(e))}{\text{valueUSD}(\text{in}(e))} \tag{1}$$

and flag edges where $|\rho(e) - 1|$ exceeds an edge-type-specific threshold. For DEX swap edges we use $\alpha = 0.05$, a conservative bound above the 1.16% average unexpected slippage on Uniswap reported by Zhou et al. [23]. For lending and oracle-mediated edges involving stablecoin derivatives we use a tighter $\beta = 0.02$, reflecting typical stablecoin oracle noise: an empirical study of DeFi oracles [11] shows that SAI and DAI daily USD price changes stay within $\leq$1% on 66–70% of days and within $\leq$5% on over 98% of days, so $\beta = 0.02$ sits just above this typical noise floor. Our rule set comprises nine DSL constraints in 107 lines, spanning AMM invariants, lending and accounting checks, oracle anomalies, bridge integrity, and flash-loan patterns. The grammar and examples appear in Appendix C. Protocol-specific logic is confined to the SFG construction layer. The constraints themselves are protocol-independent.

Evidence extraction. Given the flagged edges and the SFG, the pipeline produces replayable evidence traces with per-vertex profit and loss (PnL). An evidence trace $H \subseteq G$ is the subgraph induced by all flagged edges within a transaction. For each vertex v in H, we compute:

$$\begin{aligned} \mathrm{PnL}(v) = &\sum_{e \in \mathrm{in}_H(v)} \mathrm{valueUSD}(e) \\ &- \sum_{e \in \mathrm{out}_H(v)} \mathrm{valueUSD}(e) \end{aligned} \tag{2}$$

This highlights beneficiaries (positive PnL) and likely victims (negative PnL); we attribute impact across protocols, separating the protocol where the violation occurs from the entity that bears the loss. The per-step computation enables independent replay, addressing C3.

Illustrative example. The bZx incident [14, 15] illustrates the pipeline end to end. The attacker borrows via flash loan, manipulates a low-liquidity pool, and extracts profit from a lending protocol that trusts the manipulated price (Figure 2). The price-inflating swap yields $\rho = 1.56$ (56% excess output relative to input) and the victim's counterparty swap yields $\rho = 0.35$ (65% value loss), both triggering the abnormal-swap constraint. The evidence trace links these flagged edges to the attacker's profit path.

Validation on known incidents. We curated a confirmed-incident set of 40 publicly documented incidents with available transaction traces across our five target chains; the set spans price manipulation attacks for DeFiRanger comparison plus reentrancy, governance, and bridge exploits for scope breadth (Table 3). We define successful reconstruction as meeting three criteria: (i) the evidence trace identifies the correct manipulation step, (ii) it traces a profit path to the beneficiary, and (iii) at least one general-purpose constraint fires on the value-extracting edge. Across all 40 confirmed incidents, our pipeline meets these criteria. Table 1 summarizes incident diversity. Price-deviation and AMM constraints each fire on 55.0% of incidents, followed by reentrancy and collateral/accounting at 30.0% each.

Finding (RQ1). The nine protocol-independent rules reconstruct 40/40 confirmed incidents across five chains without per-incident logic. These rules capture structural patterns that recur across incidents, which is what makes the scale-out evaluation in §4 tractable.

Table 1. Confirmed incidents (RQ1): distribution by attack type and coverage by constraint family.

Attack type	Count	Constraint family	Incidents*
Flash loans (incl. flash-loan+price)	13	Price-deviation constraints	22 (55.0%)
Price manipulation	11	AMM constraints	22 (55.0%)
Reentrancy	7	Collateral/accounting constraints	12 (30.0%)
Oracle manipulation	4	Exchange-rate constraints	9 (22.5%)
Bridge exploits	2	Oracle integrity constraints	7 (17.5%)
Other (logic, governance, donation)	3	Bridge / flash-loan constraints	1 (2.5%) each
Total	**40**		

*Percentages sum to >100% because one incident may trigger multiple constraint families.

4 Measurement Results

We evaluate Evanesca at scale on 14,187,803 Ethereum transactions (2020–2024). The 40 confirmed incidents used in Section 3 span five chains. Section 4.1 details the dataset and collection strategy; Section 4.2 characterizes the pattern landscape at scale (RQ2); Section 4.3 validates flagged defects via source-code inspection (RQ3); and Section 4.4 compares coverage with DeFiRanger.

4.1 Data and Setup

Scope and collection. For January 2020 through December 2024, we collect all Ethereum transactions matching our 40+ adapter set (Appendix B), yielding 14,187,803 transactions. For the remaining four chains (BSC, Arbitrum, Avalanche, and Optimism), we retrieve only the transactions associated with our 40 confirmed incidents; these chains contribute RQ1 coverage but not RQ2/RQ3 scale-out. Token values are normalized to USD at each transaction's block timestamp (Appendix B). Instance counts should be interpreted as results over this Ethereum dataset, not as multi-chain population prevalence estimates.

4.2 Pattern Landscape at Scale

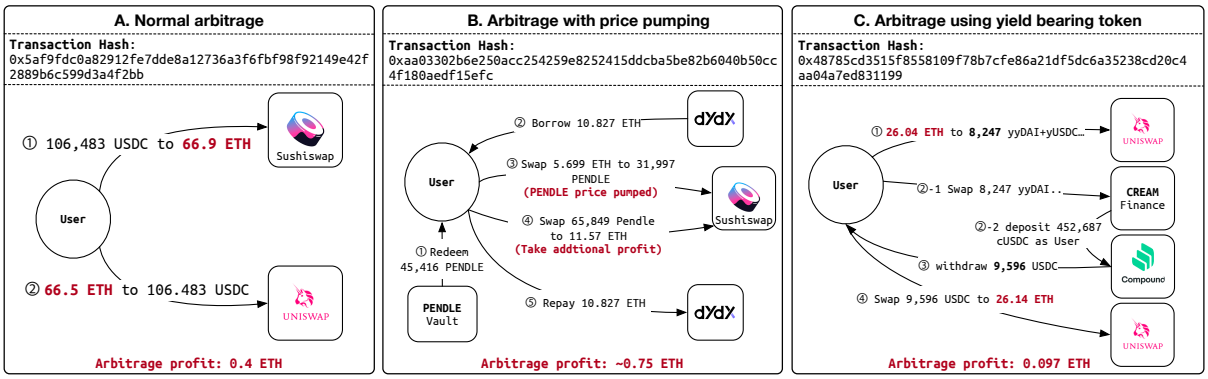

Fig. 3. Arbitrage patterns at scale (RQ2), from benign baseline (Panel A) to non-baseline variants: price pumping (B) and yield-token cross-protocol paths (C).

We flag 1,418 instances across 14,187,803 transactions using the nine DSL constraints. Table 2 summarizes the distribution across pattern categories.

Baseline separation. Of the 1,418 flagged instances, 956 are routine DEX arbitrage: they exceed $\alpha = 0.05$ but evidence traces show no flash-loan or oracle/lending constraint hit, so we retain them as benign baseline (Figure 3, Panel A). The remaining 462 non-baseline instances combine the trigger with at least one such hit, examined below. We manually validated this partition by cross-checking each instance's Etherscan event log against its SFG, consulting smart-contract source when action-level evidence was insufficient (Appendix D).

Non-baseline categories. We group the 462 non-baseline instances by extraction mechanism, yielding three named families plus a long tail. *Price manipulation* (166 instances) uses flash-loan capital to inflate a token's price on a liquidity pool, then sells previously held tokens at the inflated price on the same pool (Figure 3, Panel B). *Yield-bearing token deviation* (12 instances) reflects gaps between on-chain exchange rates and fair redemption values for derivative tokens. The profit path runs through cross-protocol deposit/withdraw cycles rather than a single corrective swap (Figure 3, Panel C). Appendix F gives step-by-step walkthroughs for Panels B and C. *Reward-distribution manipulation* (207 instances) uses flash-loan-funded momentary deposits to capture reward

shares without sustained holding (Figure 4). The remaining 77 instances form a long tail that does not cluster into the three named families above.

Table 2. Breakdown of flagged instances at scale (RQ2), including baseline.

Category	Count	Example signal
Price-discrepancy arbitrage (baseline)	956	$\lvert\rho(e) - 1\rvert$ large
Reward-distribution manipulation	207	transient in/out around reward trigger
Price manipulation	166	clustered abnormal swaps
Yield-bearing token deviation	12	persistent share-price gap
Other non-baseline patterns	77	long tail (mixed signals)
Total	**1,418**	

Finding (RQ2). At 14.2M-transaction scale, the same nine constraints flag 1,418 instances forming a measured pattern landscape: 956 routine arbitrage and 462 non-baseline transactions spanning three distinct families plus a long tail. This resolution shows that DeFi value extraction at scale is structurally diverse, not a single attack template scaled up.

4.3 Previously Unreported Implementation Defects

We inspected all 462 non-baseline instances via event-log and source-code analysis, independently of the extraction-mechanism grouping in Section 4.2. Of the 462 inspected instances, 219 have a harm-causing mechanism we identified in source code, and split into two defect families: all 12 yield-bearing cases map to derivative pricing errors and all 207 reward-manipulation cases to distribution bias. For the remaining 243 (166 price manipulation + 77 long-tail), source-code inspection revealed no implementation defect—extraction is driven by attackers exploiting otherwise correct service logic.[2] Manual inspection of all 956 baseline instances found no harmful activity (Appendix D), validating this partition as a reliable triage primitive.

Exchange-rate computation errors. Cross-protocol arbitrageurs captured exchange-rate gaps via deposit/withdraw cycles, unnoticed by protocol operators. Twelve stablecoin-backed derivative tokens (cUSDC and yyDAI families) showed 2–2.5% mispricing per instance, caused by systematic on-chain rate errors where the on-chain rate is computed from the derivative token's own contract and misses deviations at the same token's other trading venues. Our detection catches these at $\beta = 0.02$ over lending/oracle-mediated edges (Section 3).

[2]We searched DeFiHackLabs, Rekt News, SoK: DeFi Attacks [24], and audit reports for the affected tokens; neither defect family appears in prior aggregated measurement literature.

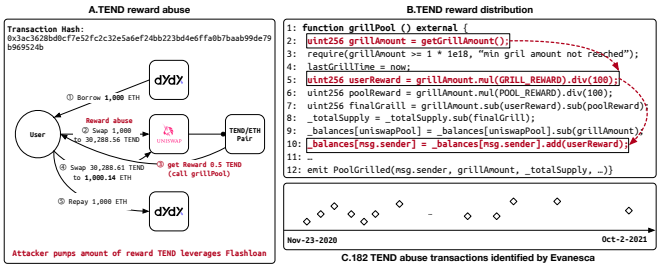

Fig. 4. Representative reward-distribution manipulation (TEND), an RQ3 case study. A flash-loan-funded swap triggers reward payout and an immediate unwind.

Reward-distribution bias. Several contracts compute payouts based on the balance at the moment of interaction rather than time-weighted holdings, enabling flash-loan-funded deposits that earn disproportionate rewards within a single transaction. Evanesca flags 207 instances, all concentrated in the TEND and WING contracts (shared deflationary logic), cumulatively yielding 15.31 ETH to attackers. Figure 4 illustrates a representative instance (TEND): a flash-loan-funded swap triggers excess reward issuance, and the attacker exits via a reverse swap. The two steps occur in different contracts, requiring cross-protocol evidence traces. Currently both TEND and WING contracts (as well as the yield-bearing wrappers in the previous subsection) are either immutable or no longer actively maintained, limiting the scope for responsible disclosure.

Finding (RQ3). Of the 462 non-baseline instances, 219 (47%) correspond to real implementation defects in two previously uncatalogued families: derivative pricing errors and reward-distribution bias. The same nine constraints thus extend from incident reconstruction (RQ1) to defect discovery, without per-defect tuning.

4.4 Comparison with Prior Detection Methods

We compare Evanesca with DeFiRanger, a closely related academic framework for detecting DeFi price manipulation attacks. As it is closed-source, we adopt DeFiScope's per-incident verdicts [22] for the 12 overlapping incidents and apply DeFiRanger's published Tr1→Ba→Tr2 pattern for the remaining 28.

Coverage results. Our constraints detect all 40. DeFiRanger covers 12 of 40 (Appendix E). The 23 out-of-scope misses cluster across reentrancy, governance, bridge, oracle defects, admin-key compromise, direct fund injection, and logic-bug mechanisms. The 5 within-scope misses—Pancake Bunny, CreamFinance #2, EGD Finance, Elephant Money, and Inverse Finance—are PMAs per [22] that DeFiRanger fails on, exposing CFT brittleness across flash-mint chains, share-price defects, and asymmetric (no-Tr2) extractions. Our approach detects all five because constraints evaluate each edge independently.

Within-scope comparison. We restrict the comparison to the 17 incidents within DeFiRanger's price manipulation scope: DeFiRanger covers 12 (70.6%) and our constraints detect 17 (100%). The gap reflects both coverage breadth and within-scope pattern brittleness. DeFiScope itself misses Harvest #1 (compilation), Indexed Finance (LLM-computation), and Inverse Finance (cross-transaction) among the 12 overlapping incidents [22]. Evanesca catches all three. We can extend Evanesca to a new protocol family by adding only a semantic adapter, without new detection templates (Appendix E).

Architectural difference. DeFiRanger builds cash-flow trees (CFTs) from raw ERC-20 token transfers and pattern-matches Transfer–Balance-change–Transfer (Tr1→Ba→Tr2) sequences, where Tr1 acquires capital, a balance change alters internal state, and Tr2 extracts value at the altered state. Unlike DeFiRanger, our approach constructs SFGs with typed financial actions such as `swap`, `deposit`, and `borrow`, and expresses detection logic as DSL constraints over these edges (Appendix C). This enables constraint categories such as reentrancy, governance,

bridge, and oracle defects that fixed transfer-level templates cannot express. This explains the 23 out-of-scope misses above: their extraction mechanism is invisible at the transfer level.

5 Related Work and Discussion

DeFi incident analysis. Prior academic systems analyze DeFi attacks through narrow representations: symbolic oracle analysis with invariant monitoring [18], asset-flow diagrams for individual flash-loan events [2], and attack parameter optimization over DeFi state [15]. More recent frameworks such as DeFort [21] and DeFiScope [22] target price manipulation attacks specifically, tailored to that single attack class. CPMMX [9] automatically synthesizes exploits for constant-product market maker (CPMM) composability bugs, but unlike our retrospective measurement, CPMMX targets exploit generation for a specific AMM family. Unlike these systems, our approach uses invariants that apply to all financial actions such as swaps, deposits, and borrows, regardless of the specific service, enabling coverage of patterns not anticipated by predefined templates.

MEV and operational monitoring. MEV research shows that significant value extraction comes from automated ordering strategies rather than software vulnerabilities [1, 7, 19]. Our patterns overlap but focus on repeatable sequences that exploit protocol mechanics rather than optimizing extraction strategies. Industry monitors such as Forta [8] and OpenZeppelin Defender [13] emphasize real-time alerting but struggle with repeated gray patterns lacking an obvious exploit trace. Our approach makes such patterns analyzable with explicit rules and replayable evidence that can be re-run across studies.

Key measurement insights. Our results yield three broader observations. First, repeatable DeFi extraction arises from how DeFi primitives such as lending, swapping, and yield generation work, rather than from how any single protocol implements them: the same small constraint set recurs across protocols, forks, and chains (Table 2). Second, our nine constraints target invariants over DeFi primitive actions; extending coverage to orthogonal behaviors, such as gas-cost anomalies or manipulation of supply mechanisms, would require a separate rule family, which we leave to future work. Third, the 309 ms mean processing time (Appendix B) permits online circuit-breaking within attack sessions.

Threats to validity. Our confirmed-incident set is curated from public post-mortem reports and may not represent all attack categories. Our semantic adapters cover major AMM, lending, flash-loan, and bridge protocols but not all DeFi primitives (e.g., options, perpetuals). Patterns in unsupported protocols would be missed. The $\alpha = 0.05$ threshold is grounded in prior findings [23], set conservatively above the 1.16% average unexpected slippage on Uniswap reported there. Because confirmed defects span multiple constraint families beyond the swap-ratio check, the RQ3 finding is not dominated by this single parameter. A formal α/β sensitivity surface is deferred to deployment-specific calibration.

Limitations. The nine constraints target invariants over DeFi primitive actions and may miss purely computational vulnerabilities (e.g., access-control bypass) that do not alter observable financial outputs. The 219 confirmed defects (47% of 462) are also a conservative floor; the remaining 243 are price manipulation attacks or long-tail patterns with no identifiable protocol code defect.

6 Conclusion

We presented Evanesca, a measurement pipeline that reconstructs Semantic Financial Graphs from on-chain activity and evaluates reusable DSL constraints over typed financial actions. Across 14,187,803 Ethereum transactions (2020–2024) and 40 confirmed incidents spanning five chains, nine constraints in 107 lines reconstruct every incident, flag 1,418 operational instances spanning five pattern categories, and surface 219 previously unreported

defects across two categories, all without per-incident tuning. These results establish Evanesca as a reusable starting point for longitudinal DeFi risk measurement.

References

[1] Crypto Briefing. 2021. What Is MEV? Ethereum's Invisible Tax Explained. https://cryptobriefing.com/what-is-mev-ethereums-invisible-tax-explained/. Accessed: 2026-04-30.

[2] Yixin Cao, Chuanwei Zou, and Xianfeng Cheng. 2021. Flashot: A Snapshot of Flash Loan Attack on DeFi Ecosystem. *arXiv preprint arXiv:2102.00626* 1 (2021), 1 – 25.

[3] ChainLink. [n. d.]. Off-chain price oracle - securely connect smart contracts with off-chain data and services. https://chain.link/. Accessed: 2026-04-20.

[4] CoinGecko. 2024. CoinGecko API: Historical Cryptocurrency Prices and Market Data. https://www.coingecko.com/en/api. Accessed: 2026-04-16.

[5] CoinMarketCap. [n. d.]. CoinMarketCap API: Historical Cryptocurrency Prices and Market Data. https://coinmarketcap.com/api/documentation/. Accessed: 2026-04-20.

[6] Josselin Feist, Gustavo Grieco, and Alex Groce. 2019. Slither: A Static Analysis Framework for Smart Contracts. In *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, Montreal, QC, Canada, 8–15.

[7] Flashbots. 2024. Flashbots: Illuminating the Dark Forest. https://www.flashbots.net/. Accessed: 2026-04-06.

[8] Forta Foundation. 2022. Forta: A Decentralized Runtime Security Solution for Automated Threat Detection and Prevention. https://docs.forta.network/en/latest/2022-7-11%20Forta%20Litepaper.pdf. Forta Litepaper. Accessed: 2026-04-06.

[9] Sujin Han, Jinseo Kim, Sung-Ju Lee, and Insu Yun. 2025. Automated Attack Synthesis for Constant Product Market Makers. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 47–68. doi:10.1145/3728872

[10] Immunefi. 2025. Crypto Losses in Q1 2025 Report. https://downloads.ctfassets.net/t3wqy70tc3bv/kicQd50NcNkEQyA1QmpOB/b365451e130c1b1c0cbe55c191a22d41/Immunefi_Crypto_Losses_In_Q1_2025.pdf. Accessed: 2026-04-21.

[11] Bowen Liu, Pawel Szalachowski, and Jianying Zhou. 2021. A First Look into DeFi Oracles. In *2021 IEEE International Conference on Decentralized Applications and Infrastructures (DAPPS)*. IEEE, 39–48.

[12] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. 2016. Making Smart Contracts Smarter. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. ACM, Vienna Austria, 254–269.

[13] OpenZeppelin. 2024. OpenZeppelin Defender: Smart Contract Monitoring with Rule-based Sentinels. https://docs.openzeppelin.com/defender/module/monitor. Accessed: 2026-04-06.

[14] Peckshield. 2021. bZx Hack Full Disclosure. https://peckshield.medium.com/bzx-hack-full-disclosure-with-detailed-profit-analysis-e6b1fa9b18fc. Accessed: 2026-04-30.

[15] Kaihua Qin, Liyi Zhou, Benjamin Livshits, and Arthur Gervais. 2021. Attacking the DeFi Ecosystem with Flash Loans for Fun and Profit. In *International Conference on Financial Cryptography and Data Security*. Springer, Springer, Grenada, 3–32.

[16] Michael Rodler, Wenting Li, Ghassan O Karame, and Lucas Davi. 2019. Sereum: Protecting Existing Smart Contracts Against Re-Entrancy Attacks. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, San Diego, CA.

[17] Petar Tsankov, Andrei Marian Dan, Dana Drachsler-Cohen, Arthur Gervais, Florian Bünzli, and Martin T. Vechev. 2018. Securify: Practical Security Analysis of Smart Contracts. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. ACM, Toronto, Canada, 67–82.

[18] Bin Wang, Han Liu, Chao Liu, Zhiqiang Yang, Qian Ren, Huixuan Zheng, and Hong Lei. 2021. BLOCKEYE: Hunting for DeFi Attacks on Blockchain. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, IEEE, Virtual, 17–20.

[19] Ben Weintraub, Christof Ferreira Torres, Cristina Nita-Rotaru, and Radu State. 2022. A Flash(bot) in the Pan: Measuring Maximal Extractable Value in Private Pools. In *Proceedings of the 22nd ACM Internet Measurement Conference (IMC)*. ACM, 458–471.

[20] Siwei Wu, Zhou Yu, Dabao Wang, Yajin Zhou, Lei Wu, Haoyu Wang, and Xingliang Yuan. 2024. DeFiRanger: Detecting DeFi Price Manipulation Attacks. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2024), 4147–4161.

[21] Maoyi Xie, Ming Hu, Ziqiao Kong, Cen Zhang, Yebo Feng, Haijun Wang, Yue Xue, Hao Zhang, Ye Liu, and Yang Liu. 2024. DeFort: Automatic Detection and Analysis of Price Manipulation Attacks in DeFi Applications. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, Vienna, Austria, 1517–1529.

[22] Juantao Zhong, Daoyuan Wu, Ye Liu, Maoyi Xie, Yang Liu, Yi Li, and Ning Liu. 2025. DeFiScope: Detecting Various DeFi Price Manipulations with LLM Reasoning. *IEEE Transactions on Dependable and Secure Computing* (2025). Available at https://ieeexplore.ieee.org/document/11334472; preprint `arXiv:2502.11521`.

[23] Liyi Zhou, Kaihua Qin, Christof Ferreira Torres, Duc Viet Le, and Arthur Gervais. 2021. High-Frequency Trading on Decentralized On-Chain Exchanges. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 428–445.

[24] Liyi Zhou, Xihan Xiong, Jens Ernstberger, Stefanos Chaliasos, Zhipeng Wang, Ye Wang, Kaihua Qin, Roger Wattenhofer, Dawn Song, and Arthur Gervais. 2023. SoK: Decentralized Finance (DeFi) Attacks. In *IEEE Symposium on Security and Privacy (SP)*. IEEE, San Francisco, CA, 2444–2461.

A Ethics

Data and privacy. This study uses only publicly available on-chain data; no private user data is collected. All blockchain transactions analyzed are permanently recorded on public ledgers and freely accessible to any observer. We do not attempt to link on-chain addresses to real-world identities, and our analysis does not enable such linking. This work does not involve human subjects and requires no IRB review.

Interpretation of flagged instances. Flagged instances are reported as measurement signals for further investigation, not as accusations of wrongdoing. We disclose only the specific transaction hashes shown in the paper's figures, each of which is already indexed on public block explorers, and do not attribute intent to any specific actor.

Responsible disclosure. The 219 previously unreported implementation defects target legacy services no longer actively maintained. No live patch channel exists for any affected contract, so a private disclosure window would neither prevent further exploitation nor protect users whose positions have long since unwound. We therefore document the findings openly so that future protocols inheriting similar designs can detect and avoid these defect classes. During analysis we cross-referenced the public security community databases (DeFiHackLabs, Rekt News, SoK: DeFi Attacks [24]) and confirmed that neither defect family had been previously reported. No new exploit code is introduced in this work. The transactions we analyze were already executed on-chain and are indexed on public block explorers.

Figure 3 case study services. The two non-baseline walkthroughs shown in Figure 3 target services that are also no longer actively maintained. Panel B illustrates a price-manipulation attack against a Pendle V1 OT-SLP position on a five-year-old Sushiswap pool; Pendle V1 has been superseded by V2 and V3. Panel C illustrates one of the 12 derivative-pricing defects discussed above and targets Cream Finance's CreamY stablecoin AMM (no active development since the protocol's 2021 security incidents; CreamY runs in withdraw-only mode).

B Implementation

Evanesca is implemented as a modular TypeScript framework with four layers mirroring the pipeline stages in Section 3: (i) *semantic recovery* (event decoding and protocol-specific action mapping), (ii) *graph construction* (SFG building with strict action ordering), (iii) *constraint evaluation* (DSL matching over SFG edges), and (iv) *evidence extraction* (subgraph induction and per-vertex PnL). The implementation goal is to keep outputs reproducible from on-chain data while keeping batch execution practical at multi-year scale. We plan to release the framework as open source.

Semantic adapters. Each supported protocol family has a dedicated adapter that maps its event schema to SFG action edges. We currently ship adapters for 40+ protocol families spanning DEX (Uniswap V1/V2/V3, SushiSwap, PancakeSwap, Curve, Balancer, Bancor, KyberSwap, Synthetix, Platypus, WooFi, and others), lending (Compound, Aave, Cream Finance, Venus, Inverse, Euler, bZx, Radiant, Hundred, and others), flash-loan (dYdX, AaveV3, DODO), and bridge protocols (Qubit, Allbridge), covering constant-product AMMs, stableswap, concentrated liquidity, lending, flash-loan, and cross-chain primitives. Adding a new protocol requires implementing one adapter module; existing constraints and evidence extraction remain unchanged.

Scalability and batch measurement. For each transaction, Evanesca evaluates all DSL constraints on every SFG edge. The current rule set contains nine constraints in 107 lines of DSL. Each constraint includes a `when` clause

that acts as a lightweight type filter; for example, a DEX invariant fires only on swap edges. Irrelevant constraints are then skipped without full evaluation. Performance comes from three optimizations: (i) DSL constraints are compiled to JavaScript functions at load time, avoiding repeated parsing; (ii) parsed constraint ASTs are cached with SHA-256 hashing; and (iii) token-to-USD conversions are batched before constraint evaluation to avoid redundant price lookups.

USD normalization. Resolved CoinGecko prices [4] are cached per (token, timestamp) pair with LRU eviction and bounded TTL to reduce API calls during batch processing. CoinGecko exposes historical prices indexed by timestamp, so the same transaction can be re-priced consistently across runs as long as the API remains accessible. Alternative historical-price sources (e.g., CoinMarketCap's historical API [5], Chainlink on-chain price feeds [3]) could replace CoinGecko without changes to the rest of the pipeline.

Deployment and throughput. Our distributed deployment uses 3 machines (Intel Xeon E5-2640 v4 @ 2.40 GHz, 128 GiB RAM each) running 30 Docker workers, 10 per machine. Mean per-transaction processing time is 309 ms (p95 $<$ 1.2 s; max 9.9 s), keeping multi-year measurement practical while preserving reproducible evidence for flagged instances.

C DSL Syntax and Example Constraints

Purpose. Evanesca encodes value-extraction pattern hypotheses as *auditable constraints* over SFG action edges (Section 3). The DSL is designed to make patterns easy to review, version, and rerun across time and services, while still producing replayable evidence artifacts.

Constraint format. Each constraint declares (i) when it applies, (ii) derived variables, and (iii) a violation predicate:

```
constraint NAME {
  when: <boolean expression over edge fields>
  condition: { x: <expr>, y: <expr>, ... }
  violation: <boolean expression>
  message: "human-readable summary"
}
```

`when` filters the edges to which the constraint applies; `condition` binds intermediate expressions by name (used by `violation`); `message` is attached to flagged matches for triage. Expressions support arithmetic and boolean operators (`+`, `-`, `*`, `/`, `<`, `<=`, `==`, `&&`, `||`) and member access such as `edge.Type` and `edge.totalInUSD`. In `condition` bindings, `||` serves as a fallback operator: for example, `x || 0` yields `x` if defined, else `0`. In `violation` expressions, `||` is logical OR.

Evaluation context. The DSL is evaluated over the per-edge records emitted by semantic recovery and USD normalization. In the examples below, `edge` refers to the current SFG edge and exposes fields such as `Type`, `Action`, `Service`, `totalInUSD`, `totalOutUSD`, and other semantics derived from logs/traces.

Representative examples. The DSL constraint set used in our evaluation contains nine constraints; we show three representative patterns below. The first is the abnormal-swap rule, which computes a value ratio `price_ratio` (equivalently, $\rho(e) = \text{valueUSD(out)}/\text{valueUSD(in)}$) and adds an auxiliary trigger that fires when a large price impact coincides with transaction atomicity, suppressing noise at scale.

```
constraint PRICE_MANIPULATION {
  when: edge.Type == "DEX" &&
        edge.Action == "Swap"
  condition: {
    price_ratio:
      (edge.totalOutUSD / edge.totalInUSD),
    price_impact:
      edge.price_impact_percent || 0,
```

```
    is_atomic:
      edge.is_atomic_transaction || false
  }
  violation:
    edge.totalOutUSD > 0 &&
    edge.totalInUSD > 0 &&
    (price_ratio > 1.05 ||
     (price_impact > 10 && is_atomic))
  message:
    "Price manipulation detected via abnormal exchange rate"
}
```

Production constraints may add additional thresholding or accounting guards; we show the core rule structure for clarity.

The next example is a concrete AMM-invariant check:

```
constraint DEX_K_INVARIANT {
  when: edge.Type == "DEX" &&
        edge.Action == "Swap"
  condition: {
    k_ratio: edge.k_after / edge.k_before
  }
  violation:
    edge.k_before > 0 &&
    edge.k_after > 0 &&
    (k_ratio > 1.01 || k_ratio < 0.99)
  message:
    "D1: K-invariant violation detected in AMM"
}
```

Finally, a lending-side accounting check:

```
constraint LENDING_COLLATERALIZATION {
  when: edge.Type == "Lending" &&
        (edge.Action == "Borrow" ||
         edge.Action == "Withdraw")
  condition: {
    collateral_ratio:
      edge.collateral_value /
      edge.borrow_amount_usd,
    health_factor:
      edge.user_collateral /
      edge.user_debt,
    profit_check:
      edge.totalOutUSD > edge.totalInUSD * 1.3
  }
  violation:
    (edge.borrow_amount_usd > 0 &&
     collateral_ratio < 1.0) ||
    (edge.user_debt > 0 &&
     health_factor < 1.0) ||
    (profit_check &&
     edge.totalInUSD > 0)
  message:
    "Under-collateralized lending position detected"
}
```

These examples illustrate how patterns are encoded as explicit, replayable rules; the full library includes additional constraints for exchange-rate anomalies, flash-loan/reentrancy patterns, and bridge integrity. For readability, we insert line breaks in the examples and may shorten long message strings; the rule logic is unchanged.

D Flagged Instance Validation

Operational measurement yields flagged instances; turning these into actionable findings requires additional validation. We use the following workflow, reproducible from public chain data: (i) re-extract the evidence trace from the transaction identifier and confirm that computed metrics such as $\rho(e)$ and PnL are stable under

re-execution; (ii) verify that implied profit exceeds fees/gas and that the value transfer is not a pure accounting artifact; (iii) verify that the operation remains profitable under different reasonable price assumptions, reducing dependence on a single price source.

Impact accounting. We report impact in units native to each extraction path rather than USD totals, avoiding dependence on volatile price sources. For exchange-rate/share-price errors, we express the per-extraction rate gap as a percentage. For reward-distribution manipulation, we compute the attacker's net profit after all borrowed capital is repaid (e.g., flash-loan repayment); the remaining tokens represent value extracted from the protocol's reward pool. These figures are conservative lower bounds and should be interpreted as measurement signals grounded in replayable evidence rather than as definitive accounting.

Manual audit of baseline instances. We manually inspected all 956 baseline instances by cross-checking each transaction's Etherscan event log against Evanesca's SFG (and smart-contract source when action-level evidence was insufficient). None met our criteria for harmful activity (price manipulation, protocol exploitation, or other adversarial behavior). All instead exhibited cross-protocol price discrepancies (large $|\rho(e) - 1|$ values) with rapid position unwinding consistent with routine arbitrage.

E Per-Incident Comparison

This appendix provides per-incident outcomes and a constraint-family breakdown for the DeFiRanger comparison summarized in §4.4. DeFiRanger is closed-source. We adopt per-incident verdicts from DeFiScope [22] for the 12 incidents in their dataset that overlap with ours, and apply DeFiRanger's published Tr1→Ba→Tr2 transfer pattern for the remaining 28, classifying each as in-scope detected, in-scope missed, or out-of-scope.

Table 3 reports the per-incident detection for both systems. We use three outcome markers for DeFiRanger:

- ✓: *in-scope detected* — the incident's transfer sequence matches DeFiRanger's published Tr1→Ba→Tr2 cash-flow-tree pattern.
- ✗: *in-scope missed* — the incident is a price manipulation, but the published pattern's structural requirements fail to fire (e.g., bZx's split capital acquisition breaks the contiguous transfer sequence).
- "–": *out-of-scope* — outside DeFiRanger's published price-manipulation focus.

Incidents whose value extraction relies on other mechanisms – reentrancy, governance, bridge accounting, share-price oracle defects, admin-key compromise, direct fund injection into pool reserves, or pure logic bugs – are marked "–" as out-of-scope. The 23 out-of-scope misses cluster into seven mechanism groups:

- Reentrancy (6): CreamFinance #1, Origin Protocol, Akropolis, Rari Capital, Curve Finance, dForce (the last via read-only reentry on Curve).
- Governance (2): BeanstalkFarms, Fortress Loans (the latter combined with oracle abuse).
- Bridge accounting (2): Qubit Finance, Allbridge.
- Share-price or oracle-misconfiguration defects (3): Yearn Finance, Radiant Capital, Concentric.
- Admin-key compromise (2): Rikkei Finance, Crosswise (oracle overwritten directly).
- Direct fund injection (2): Euler Finance, Hundred Finance (donated funds into pool reserves to inflate share price without a swap step).
- Pure logic bugs (6): KyberSwap, Platypus Finance, BlueberryFDN, Shido Global, Saddle Finance (virtual-price arithmetic), MIM Spell (cauldron precision).

The 5 within-scope misses (Pancake Bunny, CreamFinance #2, EGD Finance, Elephant Money, Inverse Finance) are discussed in §4.4.

Table 3. Scope-aware coverage comparison (RQ1): Evanesca vs. DeFiRanger's published scope mapped against our 40 confirmed DeFi incidents. DeFiRanger targets price manipulation attacks via cash-flow-tree pattern matching; attacks outside this scope are marked "–."

#	Incident	Loss	Family	Chain	Evanesca	DR	#	Incident	Loss	Family	Chain	Evanesca	DR
2020–2021													
1	CreamFinance #1	$18.8M	Collateral, Exch. rate	ETH	✓	–	8	Akropolis	$2M	Collateral, Exch. rate	ETH	✓	–
2	Harvest #1	$24M	Exch. rate	ETH	✓	✓	9	Cheese Bank	$3.3M	AMM, Price dev.	ETH	✓	✓
3	bZx Hack	$350K	AMM, Collateral, Oracle, Price dev.	ETH	✓	✓	10	Yearn Finance	$11M	Collateral, Oracle	ETH	✓	–
4	Harvest #2	$10M	Exch. rate	ETH	✓	✓	11	xToken #2	$24.5M	AMM, Price dev.	ETH	✓	✓
5	Origin Protocol	$7M	Collateral, Exch. rate	ETH	✓	–	12	Pancake Bunny	$45M	AMM, Price dev.	BSC	✓	✗
6	Value DeFi	$6M	AMM, Price dev.	ETH	✓	✓	13	CreamFinance #2	$130M	Collateral	ETH	✓	✗
7	Warp Finance	$7.7M	Price dev.	ETH	✓	✓	14	Indexed Finance	$16M	AMM, Price dev.	ETH	✓	✓
2022													
15	BeanstalkFarms	$182M	AMM, Price dev.	ETH	✓	–	21	Float Protocol	$1.4M	AMM, Price dev.	ETH	✓	✓
16	Qubit Finance	$80M	Bridge	BSC	✓	–	22	Saddle Finance	$10M	Collateral	ETH	✓	–
17	Rari Capital	$80M	Collateral, Exch. rate	ETH	✓	–	23	Fortress Loans	$3M	Collateral, Flash loan	BSC	✓	–
18	EGD Finance	$36M	AMM, Price dev.	BSC	✓	✗	24	Rikkei Finance	$1.1M	AMM, Price dev.	BSC	✓	–
19	Inverse Finance	$15.6M	AMM, Price dev.	ETH	✓	✗	25	Crosswise	$0.9M	AMM, Collateral, Exch. rate, Price dev.	BSC	✓	–
20	Elephant Money	$11.2M	AMM, Price dev.	BSC	✓	✗	26	Wiener DOGE	$0.1M	AMM, Price dev.	BSC	✓	✓
2023													
27	Euler Finance	$200M	Collateral, Exch. rate	ETH	✓	–	31	Platypus Finance	$8.5M	AMM, Price dev.	AVAX	✓	–
28	Curve Finance	$41M	AMM, Price dev.	ETH	✓	–	32	Allbridge	$0.6M	AMM	BSC	✓	–
29	KyberSwap	$48M	AMM, Price dev.	ETH	✓	–	33	dForce	$3.7M	Oracle	ARB	✓	–
30	Hundred Finance	$7M	Collateral, Exch. rate	OP	✓	–							
2024													
34	Radiant Capital	$4.5M	Oracle	ARB	✓	–	38	BlueberryFDN	$1.4M	Price dev.	ETH	✓	–
35	WooFi Swap	$8.8M	AMM, Oracle, Price dev.	ARB	✓	✓	39	Concentric	$1.8M	Oracle	ARB	✓	–
36	Gamma Strategies	$3.4M	AMM, Oracle	ARB	✓	✓	40	Shido Global	$3.3M	AMM, Price dev.	BSC	✓	–
37	MIM Spell	$6.5M	AMM, Price dev.	ETH	✓	–							
Overall												**40/40 (100%)**	**12/40 (30.0%)**
Within PMA scope												**17/17 (100%)**	**12/17 (70.6%)**

F Arbitrage Pattern Walkthroughs

This appendix provides step-by-step reconstructions of representative instances for Figure 3 Panels B and C. Each walkthrough is derived from publicly indexed on-chain transactions and is stated in past tense; the flagged constraint(s) appear at the end of each case.

Panel B: Price Manipulation. In this pattern the attacker uses flash-loan capital to inflate a token's price on a single liquidity pool, then sells a pre-existing token position into the inflated price before closing the loan. The example concerns a yield-bearing token (PENDLE) traded on Sushiswap.

(1) An arbitrageur borrowed 10.827 ETH as a flash loan from dYdX.
(2) The arbitrageur bought 31,997 PENDLE from Sushiswap using 5.699 ETH, inflating PENDLE's Sushiswap price.
(3) The arbitrageur then sold 65,849 pre-existing PENDLE back to Sushiswap at a price roughly 10% above the pre-attack level.
(4) The arbitrageur repaid the 10.827 ETH flash loan to dYdX. Net gain: approximately 0.75 ETH per atomic transaction.
Fires: `PRICE_MANIPULATION` (abnormal $\rho(e)$ on the inflated swap) and `FLASH_LOAN_ATTACK` (atomic borrow–manipulate–repay cycle).

Panel C: Yield-Bearing Token Deviation. In this pattern the attacker routes a yield-bearing vault share through a stablecoin AMM that mispricess share tokens, then redeems the AMM's released collateral at the underlying protocol's fair rate. The example involves Cream Finance's CreamY pool, which held Compound cUSDC as a reserve alongside Yearn's "yyDAI+yUSDC+yUSDT+yTUSD" vault share, but priced the vault share by raw pool balance rather than by its Yearn per-share value.

(1) An arbitrageur bought 8,247 "yyDAI+yUSDC+yUSDT+yTUSD" tokens on Uniswap using 26.04 ETH.

(2) The arbitrageur deposited these tokens into Cream Finance's CreamY pool, whose Balancer-style invariant treated the yield-bearing share as a 1:1 stablecoin. CreamY released 452,687 Compound cUSDC in exchange, roughly 2.0% above fair value for the share token.
(3) The arbitrageur redeemed 452,687 cUSDC on Compound at Compound's then-current exchange rate, receiving 9,596 USDC.
(4) The arbitrageur swapped 9,596 USDC to 26.14 ETH.
(5) Net gain: approximately 0.097 ETH per cycle, extracted from CreamY's stale-share-price accounting.
Fires: `EXCHANGE_RATE_MANIPULATION` at the Compound cUSDC redeem edge (step 3): the cUSDC position acquired through the prior CreamY swap (step 2) is unwrapped at Compound's fair per-share rate, and the redeem-vs-acquisition USD ratio departs from 1.0 by more than $\alpha = 0.05$, exposing the cross-protocol exchange-rate gap as a same-transaction extraction cycle (cf. §3, §4.2 'cross-protocol deposit/withdraw cycles').